

Perfecto. Con ese stack armamos un ecommerce moderno y profesional:

- **Frontend:** Next.js + TypeScript
- **Estilos:** Tailwind CSS
- **Data fetching / cache:** React Query
- **Backend:** ASP.NET Core Web API
- **DB:** PostgreSQL
- **Infra:** Docker

Ese stack es muy bueno porque te obliga a aprender arquitectura real de frontend, backend, base de datos y despliegue con contenedores.

Objetivo

Construir un **ecommerce completo**, paso a paso, aplicando:

- buenas prácticas
 - arquitectura limpia
 - tipado fuerte
 - separación de responsabilidades
 - validaciones
 - autenticación
 - manejo de errores
 - Docker
 - escalabilidad
-

Arquitectura general

Frontend

Next.js + TypeScript + Tailwind + React Query

Responsabilidades:

- UI pública del ecommerce
- panel admin
- autenticación del usuario
- consumo de API
- manejo de cache, loading y errores
- formularios

Backend

ASP.NET Core Web API

Responsabilidades:

- lógica de negocio
- autenticación JWT
- validaciones
- acceso a datos
- gestión de productos, usuarios, órdenes, stock

Base de datos

PostgreSQL

Responsabilidades:

- persistencia de usuarios
- productos
- categorías
- carritos
- órdenes
- pagos

Infra

Docker + Docker Compose

Responsabilidades:

- levantar frontend
 - backend
 - base de datos
 - entorno consistente de desarrollo
-

Stack detallado recomendado

Frontend

- Next.js App Router
- TypeScript
- Tailwind CSS
- TanStack Query
- Axios

- React Hook Form
- Zod
- Zustand o Context solo si hace falta estado local/global adicional
- next/image
- next/navigation

Backend

- ASP.NET Core 8 Web API
- Entity Framework Core
- Npgsql provider
- FluentValidation
- AutoMapper
- JWT Bearer
- Swagger
- Serilog
- BCrypt para passwords

Infra

- Docker
 - Docker Compose
-

Qué vamos a construir

Módulos del ecommerce

Público

- Home
- listado de productos
- detalle de producto
- búsqueda
- filtros por categoría/precio
- carrito
- checkout
- login/registro
- perfil
- historial de órdenes

Admin

- dashboard
 - CRUD de productos
 - CRUD de categorías
 - gestión de órdenes
 - control de stock
 - gestión de usuarios
-

Entidades principales

Backend / DB

- User
 - Role
 - Category
 - Product
 - ProductImage
 - Cart
 - CartItem
 - Order
 - OrderItem
 - Address
 - Payment
-

Estructura recomendada del proyecto

Monorepo simple

```
ecommerce/  
├── frontend/  
├── backend/  
├── docker-compose.yml  
├── .env  
└── README.md
```

Frontend: estructura recomendada

Con Next.js App Router:

```
frontend/
├── src/
│   ├── app/
│   │   ├── (public)/
│   │   │   ├── page.tsx
│   │   │   ├── products/
│   │   │   ├── cart/
│   │   │   ├── login/
│   │   │   └── register/
│   │   ├── admin/
│   │   ├── profile/
│   │   ├── layout.tsx
│   │   └── globals.css
│   ├── components/
│   │   ├── ui/
│   │   ├── shared/
│   │   ├── product/
│   │   ├── cart/
│   │   └── forms/
│   ├── features/
│   │   ├── auth/
│   │   ├── products/
│   │   ├── categories/
│   │   ├── cart/
│   │   └── orders/
│   ├── lib/
│   │   ├── api/
│   │   ├── utils/
│   │   ├── constants/
│   │   └── validations/
│   ├── hooks/
│   ├── providers/
│   ├── types/
│   └── middleware.ts
├── public/
├── package.json
└── Dockerfile
```

Buenas prácticas frontend

- separar componentes visuales de lógica
- usar tipos bien definidos
- centralizar cliente HTTP
- usar React Query para server state
- usar Zod + React Hook Form para formularios
- mantener rutas y layouts ordenados
- evitar lógica pesada dentro de páginas

- crear hooks por feature
-

Backend: estructura recomendada

Te recomiendo una arquitectura por capas:

```
backend/
├── src/
│   ├── Ecommerce.API/
│   ├── Ecommerce.Application/
│   ├── Ecommerce.Domain/
│   └── Ecommerce.Infrastructure/
├── tests/
│   ├── Ecommerce.UnitTests/
│   └── Ecommerce.IntegrationTests/
├── Ecommerce.sln
└── Dockerfile
```

Responsabilidad por capa

Ecommerce.Domain

- entidades
- enums
- interfaces
- reglas del dominio

Ecommerce.Application

- DTOs
- servicios
- casos de uso
- validaciones
- interfaces de aplicación

Ecommerce.Infrastructure

- DbContext
- configuraciones EF
- repositorios
- servicios externos
- autenticación
- persistencia

Ecommerce.API

- controllers
 - middlewares
 - configuración DI
 - auth
 - swagger
-

Base de datos inicial

Tablas principales

roles

- id
- name

users

- id
- first_name
- last_name
- email
- password_hash
- role_id
- created_at
- is_active

categories

- id
- name
- slug
- description
- is_active

products

- id
- name
- slug
- description
- price

- stock
- category_id
- is_active
- created_at

product_images

- id
- product_id
- image_url
- is_main

carts

- id
- user_id

cart_items

- id
- cart_id
- product_id
- quantity
- unit_price

orders

- id
- user_id
- status
- total
- created_at
- shipping_address
- payment_method

order_items

- id
 - order_id
 - product_id
 - product_name
 - quantity
 - unit_price
 - subtotal
-

Roadmap correcto de desarrollo

Fase 1. Base del proyecto

- crear monorepo
- crear frontend con Next.js + TS
- crear backend con ASP.NET Core
- configurar PostgreSQL
- configurar Docker Compose
- conectar backend con DB
- crear migración inicial

Fase 2. Módulo catálogo

- categorías
- productos
- listado
- detalle
- búsqueda
- filtros
- paginación

Fase 3. Autenticación

- registro
- login
- JWT
- roles
- rutas protegidas en frontend
- protección de endpoints

Fase 4. Carrito

- agregar al carrito
- quitar
- actualizar cantidad
- validar stock
- persistencia

Fase 5. Checkout y órdenes

- formulario de dirección
- creación de orden

- descuento de stock
- historial de compras

Fase 6. Admin

- CRUD productos
- CRUD categorías
- órdenes
- usuarios
- stock

Fase 7. Infra y calidad

- Docker completo
- logs
- manejo global de errores
- validaciones
- testing
- deploy

Cómo usar React Query correctamente en este proyecto

React Query lo vamos a usar para todo lo que sea **server state**:

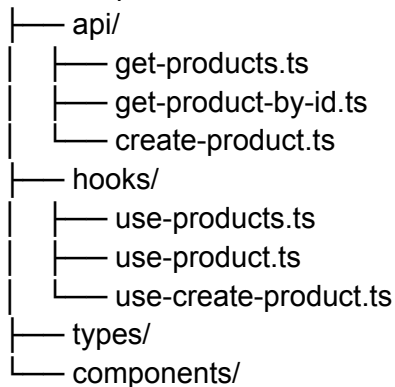
- productos
- categorías
- detalle de producto
- carrito sincronizado
- perfil
- órdenes
- admin

Patrones recomendados

- `useQuery` para lecturas
- `useMutation` para escritura
- invalidación de queries después de crear/editar/eliminar
- query keys centralizadas
- manejo de loading/error desde hooks

Ejemplo de organización

features/products/



Buenas prácticas clave

Frontend

- tipos separados por dominio
- no mezclar componentes de UI con llamadas HTTP
- usar server/client components con criterio
- usar React Query solo en client components
- formularios con RHF + Zod
- layouts reutilizables
- skeleton/loading/error states

Backend

- controllers finos
- lógica en servicios
- DTOs de entrada y salida
- no exponer entidades directamente
- validaciones con FluentValidation
- configuración por extensión
- middleware global de errores
- logging estructurado
- soft delete cuando corresponda

Base de datos

- nombres consistentes
- índices en email, slug, foreign keys
- constraints

- migraciones ordenadas
- datos semilla para roles/admin

Docker

- contenedores separados
 - variables de entorno
 - volúmenes para DB
 - redes internas
 - compose para levantar todo con un comando
-

Arquitectura de autenticación

Backend

- login con email/password
- password hasheada con BCrypt
- JWT access token
- rol en claims

Frontend

Hay dos caminos, pero para aprender bien y no complicarnos al inicio te recomiendo:

- guardar token de forma controlada
- usar interceptores en Axios
- proteger rutas de admin y perfil

Más adelante se puede evolucionar a refresh token y cookies httpOnly.

Docker: servicios iniciales

En `docker-compose.yml` vamos a levantar:

- `frontend`
- `backend`
- `postgres`

Flujo

- frontend consume backend
 - backend consume postgres
 - todo dentro de la misma red docker
-

Orden ideal para construirlo conmigo

Módulo 1

Estructura inicial del proyecto

- monorepo
- frontend base
- backend base
- docker compose
- conexión a PostgreSQL

Módulo 2

Diseño del backend

- Domain
- Application
- Infrastructure
- API
- entidades iniciales
- DbContext
- migración

Módulo 3

Productos y categorías

- backend CRUD
- frontend listado/detalle
- React Query
- filtros básicos

Módulo 4

Autenticación

- register/login
- JWT
- rutas protegidas
- perfil

Módulo 5

Carrito

- agregar/quitar/editar
- persistencia
- validación

Módulo 6

Checkout y órdenes

- crear orden
- stock
- historial

Módulo 7

Admin

- dashboard
- ABM productos/categorías
- órdenes
- usuarios

Módulo 8

Docker final + mejoras

- optimización
- logs
- testing
- deploy

Mi recomendación concreta

Para aprender bien, lo hagamos así:

Primera versión

- catálogo
- auth
- carrito
- checkout simple
- admin básico

Segunda versión

- imágenes
 - pagos
 - cupones
 - reviews
 - dashboard más completo
-

Siguiente paso

Arranquemos con el **Módulo 1** y te doy:

- estructura completa de carpetas
- comandos de creación
- `docker-compose.yml`
- configuración inicial de Next.js
- configuración inicial de ASP.NET Core
- conexión con PostgreSQL

FRONTEND — TODO lo que vas a aprender

Con **Next.js + TypeScript + React Query + Tailwind**, tocás casi todo lo moderno:

◆ Fundamentos sólidos

- Componentes
- Props
- Hooks (`useState`, `useEffect`)
- Custom hooks
- Renderizado condicional
- Listas dinámicas

◆ Next.js (MUY importante)

- App Router
- Server vs Client Components
- Layouts
- Routing dinámico (`/products/[id]`)
- Metadata SEO
- SSR / SSG / ISR
- Middleware

◆ TypeScript real

- Tipos por dominio (`Product`, `User`, etc.)
- Tipos de API (request/response)
- Generics
- Tipado estricto en formularios y hooks

◆ Manejo de datos (clave)

- React Query:
 - `useQuery`
 - `useMutation`
 - cache
 - invalidaciones
 - refetch automático
- Estados:
 - loading
 - error

- success

◆ Formularios profesionales

- React Hook Form
- Validaciones con Zod
- Manejo de errores de backend
- UX de formularios

◆ UI/UX real

- Tailwind
- Componentes reutilizables
- Responsive design
- Skeleton loaders
- Empty states
- Feedback visual

◆ Arquitectura frontend

- Separación por features
- Servicios ([api/](#))
- Hooks reutilizables
- Componentes desacoplados

◆ Autenticación

- Manejo de JWT
- Protección de rutas
- Persistencia de sesión

◆ Performance

- Lazy loading
- Optimización de imágenes ([next/image](#))
- Cacheo con React Query

 **BACKEND — TODO lo que vas a aprender**

Con **ASP.NET Core + PostgreSQL**, cubrás backend profesional real:

◆ Fundamentos backend

- APIs REST
- Controllers
- Rutas
- HTTP (GET, POST, PUT, DELETE)

◆ Arquitectura (MUY importante)

- Clean Architecture
- Separación por capas:
 - Domain
 - Application
 - Infrastructure
 - API

◆ Entity Framework Core

- DbContext
- Relaciones
- Migraciones
- Queries
- Includes
- Tracking / NoTracking

◆ Base de datos (PostgreSQL)

- Modelado relacional
- Claves foráneas
- Índices
- Normalización
- Transacciones

◆ DTOs y mapeo

- Separar entidades de respuesta
- AutoMapper
- Seguridad de datos

◆ Validaciones

- FluentValidation

- Validación de negocio
- Validación de entrada

♦ Autenticación y seguridad

- JWT
- Hash de contraseñas (BCrypt)
- Claims
- Roles
- Autorización por endpoint

♦ Lógica de negocio real

- Carrito
- Órdenes
- Stock
- Validaciones complejas

♦ Buenas prácticas

- Controllers finos
- Servicios
- Manejo de errores global
- Logging
- Configuración por ambiente

♦ Testing (si lo agregás)

- Unit tests
- Integration tests

FULLSTACK — LO MÁS IMPORTANTE

Esto es lo que realmente te hace subir de nivel:

♦ Comunicación frontend-backend

- APIs reales
- manejo de errores

- contratos tipados

◆ Flujo completo de negocio

Ejemplo real:

Usuario → agrega al carrito → checkout → orden → stock ↓ → respuesta UI

◆ Manejo de estado distribuido

- frontend (React Query)
- backend (DB)
- sincronización

◆ Seguridad

- validación en frontend ≠ confiar en frontend
- backend valida todo

◆ Persistencia

- base de datos real
- relaciones
- integridad

DEVOPS / INFRA

Con Docker aprendés:

- Contenedores
 - Redes
 - Volúmenes
 - Variables de entorno
 - Entornos reproducibles
 - Levantar todo con 1 comando
-

! LO QUE NO CUBRE COMPLETAMENTE (pero podés agregar después)

Para ser 100% senior-level, hay cosas extra:

♦ **Avanzado frontend**

- WebSockets (tiempo real)
- Microfrontends
- Animaciones complejas
- Accesibilidad profunda (a11y)

♦ **Avanzado backend**

- CQRS
- MediatR
- Event-driven architecture
- Microservicios
- Caching con Redis

♦ **Infra avanzada**

- Kubernetes
- CI/CD pipelines
- Observabilidad (Prometheus, Grafana)

♦ **Seguridad avanzada**

- Refresh tokens
- OAuth (Google, etc.)
- rate limiting